



UNIVERSITÉ GRENOBLE ALPES

COMPTE RENDU

TP3 : Construction d'un perceptron multicouche

UEX Machine Learning

Rédigé par :

DJEBBAR Anis
AKLI Mélissa
BARROUG Abdellah
OULD BRAHIM Meriem Lyna

Année Universitaire 2024-2025

Table des matières

1	Introduction et objectifs	2
2	Exploration et préparation des données	2
2.1	Jeu de données make_moons	2
3	Approches de classification	2
3.1	Classificateur linéaire manuel	2
3.1.1	Définition du classificateur	2
3.1.2	Évaluation des performances	2
3.2	Régression logistique	3
4	Implémentation d'un perceptron multicouche (MLP)	3
4.1	Architecture du réseau	3
4.2	Initialisation du modèle	4
4.3	Propagation avant (Forward Pass)	4
4.4	Fonction de perte	4
4.5	Rétropropagation du gradient (Backpropagation)	5
4.5.1	Dérivée de la fonction de perte par rapport à la sortie	5
4.5.2	Dérivée de la fonction sigmoid	7
4.5.3	Calcul des gradients pour la mise à jour des poids	8
4.6	Dimensions des matrices et vecteurs	8
4.7	Entraînement du modèle	9
4.8	Prédiction	9
5	Expérimentations et résultats	9
5.1	Influence du nombre d'époques	9
5.2	Comparaison des différentes approches	10
6	Adaptation du modèle pour la classification du jeu MNIST	11
6.1	Adaptation de l'architecture	11
6.2	Fonctions d'activation	11
6.3	Problèmes de performance et ajustements	11
6.4	Comparaison avec le modèle sur Make_Moons	11
7	Pistes d'amélioration	11
8	Conclusion	12

1 Introduction et objectifs

Ce TP est consacré à l'implémentation et l'analyse de différentes techniques de classification sur le jeu de données synthétique `make_moons`. Nous avons exploré plusieurs approches, partant de la classification linéaire simple jusqu'à la construction manuelle d'un perceptron multicouche (MLP).

Objectifs principaux

- Comprendre les limites des classificateurs linéaires
- Implémenter un réseau de neurones multicouche à partir de zéro
- Comparer les performances des différentes méthodes
- Appliquer notre modèle à un jeu de données réel (MNIST)

2 Exploration et préparation des données

2.1 Jeu de données `make_moons`

Le jeu de données `make_moons` de scikit-learn génère deux classes disposées en forme de demi-lunes imbriquées. Nous l'avons généré avec les paramètres suivants :

Code de génération

```
X, y = sklearn.datasets.make_moons(n_samples=200, noise=0.1, random_state=42)
```

Nous avons ensuite divisé ces données en ensembles d'entraînement et de test :

```
Xtrain, Xtest, ytrain, ytest = train_test_split(X, y, test_size=0.4, random_state=42)
```

Ce jeu de données bidimensionnel peut être interprété comme représentant deux classes de patients (sains et malades) caractérisés par deux mesures médicales (x_1 et x_2).

3 Approches de classification

3.1 Classificateur linéaire manuel

Notre première approche a consisté à trouver manuellement une frontière linéaire (une droite) séparant au mieux les deux classes.

3.1.1 Définition du classificateur

Nous avons implémenté une fonction `pred_linear` qui classe un point (x_1, x_2) en fonction de sa position par rapport à une droite d'équation $y = ax_1 + b$:

- Si $ax_1 + b > x_2$, le point est classé dans la catégorie 1
- Sinon, le point est classé dans la catégorie 0

3.1.2 Évaluation des performances

Pour évaluer la qualité de notre classification, nous avons utilisé la métrique d'accuracy :

$$\text{Accuracy} = \frac{\text{Nombre d'exemples correctement classifiés}}{\text{Nombre total d'exemples}} \quad (1)$$

Après plusieurs essais manuels en ajustant les paramètres a et b , nous avons obtenu une accuracy maximale d'environ 84%. Ce résultat illustre clairement les limites d'une frontière linéaire ajustée manuellement pour séparer des données non linéairement séparables.

3.2 Régression logistique

La régression logistique est un modèle de classification plus sophistiqué qui modélise la probabilité d'appartenance à une classe :

$$P(y = 1|x) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} \quad (2)$$

Nous avons utilisé l'implémentation de scikit-learn :

```
from sklearn.linear_model import LogisticRegression
classiflier = LogisticRegression()
classiflier.fit(X, y)
```

Malgré cette approche plus avancée, l'accuracy obtenue n'a pas dépassé 85%. Ce résultat confirme que même des classificateurs linéaires sophistiqués ne peuvent pas résoudre efficacement des problèmes non linéairement séparables.

4 Implémentation d'un perceptron multicouche (MLP)

4.1 Architecture du réseau

Notre MLP comporte trois couches :

- Couche d'entrée : 2 neurones (pour les caractéristiques x_1 et x_2)
- Couche cachée : 10 neurones avec fonction d'activation sigmoïde
- Couche de sortie : 2 neurones (pour les deux classes) avec fonction d'activation softmax

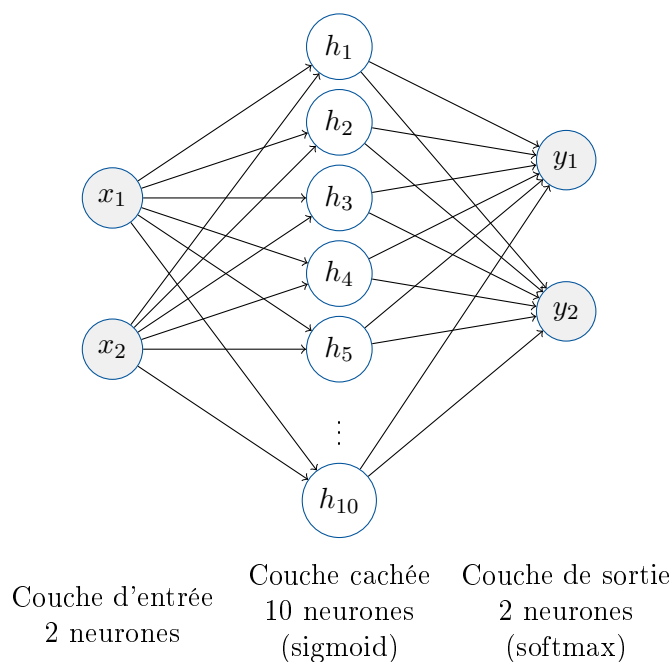


FIGURE 1 – Architecture du perceptron multicouche implémenté

4.2 Initialisation du modèle

Nous avons initialisé les poids et biais aléatoirement dans l'intervalle $[-0.5, 0.5]$:

- $W^{(1)}$: matrice de poids de dimension 2×10 (entrée $\rightarrow$ couche cachée)
- $b^{(1)}$: vecteur de biais de dimension 1×10 pour la couche cachée
- $W^{(2)}$: matrice de poids de dimension 10×2 (couche cachée $\rightarrow$ sortie)
- $b^{(2)}$: vecteur de biais de dimension $1/\text{times2}$ pour la couche de sortie

4.3 Propagation avant (Forward Pass)

La propagation avant se déroule en plusieurs étapes :

$$Z^{(1)} = X \times W^{(1)} + B^{(1)} \quad (3)$$

$$A^{(1)} = \sigma(Z^{(1)}) = \frac{1}{1 + \exp(-Z^{(1)})} \quad (4)$$

$$Z^{(2)} = A^{(1)} \times W^{(2)} + B^{(2)} \quad (5)$$

$$A^{(2)} = \text{softmax}(Z^{(2)}) \quad (6)$$

où la fonction softmax est définie par :

$$\text{softmax}(z_j) = \frac{e^{z_j}}{\sum_k e^{z_k}} \quad (7)$$

z_j est une composante du vecteur Z .

4.4 Fonction de perte

Nous avons utilisé l'entropie croisée comme fonction de perte :

$$\ell_i(y, \hat{y}) = - \sum_{c=1}^C y_c \log \hat{y}_c \quad (8)$$

où $C = 2$ est le nombre de classes, $y_{i,c}$ est l'encodage one-hot de la vraie classe, et $\hat{y}_{i,c}$ est la probabilité prédite pour la classe c , le produit est terme à terme et non matriciel.

4.5 Rétropropagation du gradient (Backpropagation)

4.5.1 Dérivée de la fonction de perte par rapport à la sortie

Pour calculer $\Delta^{(2)} = \frac{\partial \ell_i}{\partial A^{(2)}} \times \frac{\partial A^{(2)}}{\partial Z^{(2)}} = A^{(2)} - y_i x$, nous procédons comme suit :

Étape 1 : Calcul de $\frac{\partial \ell_i}{\partial A^{(2)}}$

La fonction de perte est l'entropie croisée :

$$\ell_i = - \sum_{c=1}^C y_{i,c} \log(A_{i,c}^{(2)}) \quad (9)$$

La dérivée par rapport à $A_{i,j}^{(2)}$ est :

$$\frac{\partial \ell_i}{\partial A_{i,j}^{(2)}} = - \frac{y_{i,j}}{A_{i,j}^{(2)}} \quad (10)$$

Étape 2 : Calcul de $\frac{\partial A^{(2)}}{\partial Z^{(2)}}$

$A^{(2)}$ est obtenu en appliquant la fonction softmax à $Z^{(2)}$:

$$A_{i,j}^{(2)} = \frac{e^{Z_{i,j}^{(2)}}}{\sum_{k=1}^C e^{Z_{i,k}^{(2)}}} \quad (11)$$

Pour calculer la dérivée, nous distinguons deux cas :

Cas 1 : Lorsque $j = l$ (même indice) :

$$\frac{\partial A_{i,j}^{(2)}}{\partial Z_{i,l}^{(2)}} = \frac{\partial}{\partial Z_{i,l}^{(2)}} \left(\frac{e^{Z_{i,j}^{(2)}}}{\sum_{k=1}^C e^{Z_{i,k}^{(2)}}} \right) \quad (12)$$

$$= \frac{e^{Z_{i,j}^{(2)}} \times \sum_{k=1}^C e^{Z_{i,k}^{(2)}} - e^{Z_{i,j}^{(2)}} \times e^{Z_{i,l}^{(2)}}}{(\sum_{k=1}^C e^{Z_{i,k}^{(2)}})^2} \quad (13)$$

$$= A_{i,j}^{(2)} \times (1 - A_{i,l}^{(2)}) \quad (14)$$

Cas 2 : Lorsque $j \neq l$ (indices différents) :

$$\frac{\partial A_{i,j}^{(2)}}{\partial Z_{i,l}^{(2)}} = \frac{\partial}{\partial Z_{i,l}^{(2)}} \left(\frac{e^{Z_{i,j}^{(2)}}}{\sum_{k=1}^C e^{Z_{i,k}^{(2)}}} \right) \quad (15)$$

$$= \frac{0 - e^{Z_{i,j}^{(2)}} \times e^{Z_{i,l}^{(2)}}}{(\sum_{k=1}^C e^{Z_{i,k}^{(2)}})^2} \quad (16)$$

$$= -A_{i,j}^{(2)} \times A_{i,l}^{(2)} \quad (17)$$

Ces deux cas peuvent être résumés avec le delta de Kronecker δ_{jl} :

$$\frac{\partial A_{i,j}^{(2)}}{\partial Z_{i,l}^{(2)}} = A_{i,j}^{(2)} (\delta_{jl} - A_{i,l}^{(2)}) \quad (18)$$

Étape 3 : Combinaison des résultats

En utilisant la règle de dérivation en chaîne :

$$\Delta_{i,l}^{(2)} = \sum_{j=1}^C \frac{\partial \ell_i}{\partial A_{i,j}^{(2)}} \times \frac{\partial A_{i,j}^{(2)}}{\partial Z_{i,l}^{(2)}} \quad (19)$$

$$= \sum_{j=1}^C -\frac{y_{i,j}}{A_{i,j}^{(2)}} \times A_{i,j}^{(2)} (\delta_{jl} - A_{i,l}^{(2)}) \quad (20)$$

$$= -\sum_{j=1}^C y_{i,j} (\delta_{jl} - A_{i,l}^{(2)}) \quad (21)$$

$$= -\sum_{j=1}^C y_{i,j} \delta_{jl} + \sum_{j=1}^C y_{i,j} A_{i,l}^{(2)} \quad (22)$$

$$= -y_{i,l} + A_{i,l}^{(2)} \sum_{j=1}^C y_{i,j} \quad (23)$$

$$(24)$$

Comme y_i est un vecteur one-hot, $\sum_{j=1}^C y_{i,j} = 1$, donc :

$$\Delta_{i,l}^{(2)} = -y_{i,l} + A_{i,l}^{(2)} = A_{i,l}^{(2)} - y_{i,l} \quad (25)$$

Ou sous forme vectorielle :

$$\Delta^{(2)} = A^{(2)} - y_i \quad (26)$$

4.5.2 Dérivée de la fonction sigmoid

Pour prouver que $\frac{\partial A^{(1)}}{\partial Z^{(1)}} = A^{(1)} \odot (1 - A^{(1)})$, nous partons de la définition de la fonction sigmoid :

$$A^{(1)} = \sigma(Z^{(1)}) = \frac{1}{1 + e^{-Z^{(1)}}} \quad (27)$$

La dérivée de $\sigma(z)$ par rapport à z est :

$$\frac{d\sigma(z)}{dz} = \frac{d}{dz} \left(\frac{1}{1 + e^{-z}} \right) \quad (28)$$

$$= \frac{e^{-z}}{(1 + e^{-z})^2} \quad (29)$$

$$(30)$$

Nous pouvons réécrire cette expression en fonction de $\sigma(z)$:

$$\frac{d\sigma(z)}{dz} = \frac{e^{-z}}{(1 + e^{-z})^2} \quad (31)$$

$$= \frac{1}{1 + e^{-z}} \times \frac{e^{-z}}{1 + e^{-z}} \quad (32)$$

$$= \frac{1}{1 + e^{-z}} \times \left(1 - \frac{1}{1 + e^{-z}} \right) \quad (33)$$

$$= \sigma(z) \times (1 - \sigma(z)) \quad (34)$$

En appliquant ce résultat à chaque élément de $Z^{(1)}$, nous obtenons :

$$\frac{\partial A_i^{(1)}}{\partial Z_i^{(1)}} = A_i^{(1)} \times (1 - A_i^{(1)}) \quad (35)$$

Comme la fonction sigmoïde est appliquée élément par élément, la matrice jacobienne $\frac{\partial A^{(1)}}{\partial Z^{(1)}}$ est diagonale. En notation vectorielle avec le produit de Hadamard $\odot$, nous avons :

$$\frac{\partial A^{(1)}}{\partial Z^{(1)}} = A^{(1)} \odot (1 - A^{(1)}) \quad (36)$$

4.5.3 Calcul des gradients pour la mise à jour des poids

À partir de ces dérivées, nous pouvons calculer les gradients pour mettre à jour les poids et biais :

$$\Delta^{(1)} = (\Delta^{(2)} \times (W^{(2)})^T) \odot (A^{(1)} \odot (1 - A^{(1)})) \quad (37)$$

$$\frac{\partial \ell_i}{\partial W^{(2)}} = (A^{(1)})^T \times \Delta^{(2)} \quad (38)$$

$$\frac{\partial \ell_i}{\partial B^{(2)}} = \Delta^{(2)} \quad (39)$$

$$\frac{\partial \ell_i}{\partial W^{(1)}} = X^T \times \Delta^{(1)} \quad (40)$$

$$\frac{\partial \ell_i}{\partial B^{(1)}} = \Delta^{(1)} \quad (41)$$

4.6 Dimensions des matrices et vecteurs

Pour vérifier la cohérence de nos calculs, nous avons analysé les dimensions de toutes les matrices et vecteurs :

Variable	Dimension
X	$(1, 2)$
$W^{(1)}$	$(2, 10)$
$b^{(1)}$	$(1, 10)$
$Z^{(1)}$	$(1, 10)$
$A^{(1)}$	$(1, 10)$
$W^{(2)}$	$(10, 2)$
$b^{(2)}$	$(1, 2)$
$Z^{(2)}$	$(1, 2)$
$A^{(2)}$	$(1, 2)$
$\Delta^{(2)}$	$(1, 2)$
$\Delta^{(1)}$	$(1, 10)$

TABLE 1 – Dimensions des matrices et vecteurs du réseau

Ces dimensions sont cohérentes avec les opérations matricielles effectuées dans le réseau.

4.7 Entraînement du modèle

Pour l'entraînement, nous avons implémenté une descente de gradient stochastique :

1. Initialisation aléatoire des poids et biais
2. Pour chaque époque :
 - (a) Mélange aléatoire des données d'entraînement
 - (b) Pour chaque exemple (X_i, y_i) :
 - i. Propagation avant pour calculer les activations
 - ii. Calcul de la fonction de perte
 - iii. Rétropropagation pour calculer les gradients
 - iv. Mise à jour des poids et biais :

$$W^{(2)} = W^{(2)} - \epsilon \cdot \frac{\partial \ell_i}{\partial W^{(2)}} \quad (42)$$

$$b^{(2)} = b^{(2)} - \epsilon \cdot \frac{\partial \ell_i}{\partial b^{(2)}} \quad (43)$$

$$W^{(1)} = W^{(1)} - \epsilon \cdot \frac{\partial \ell_i}{\partial W^{(1)}} \quad (44)$$

$$b^{(1)} = b^{(1)} - \epsilon \cdot \frac{\partial \ell_i}{\partial b^{(1)}} \quad (45)$$

où ϵ est le taux d'apprentissage, fixé à 0.1.

4.8 Prédiction

Une fois le modèle entraîné, nous avons implémenté une fonction de prédiction qui :

1. Effectue une propagation avant pour calculer les probabilités
2. Retourne la classe ayant la probabilité la plus élevée (argmax)

5 Expérimentations et résultats

5.1 Influence du nombre d'époques

Nous avons testé notre MLP avec différents nombres d'époques : $\{1, 15, 30, 45, 60, 75, 90, 105, 120, 135, 150\}$. Les résultats montrent une amélioration progressive de l'accuracy avec l'augmentation du nombre d'époques :

Nombre d'époques	Accuracy
1	$\approx 80\%$
15	$\approx 87\%$
45	$\approx 85.5\%$
90	$\approx 88.5\%$
150	100%

TABLE 2 – Évolution de l'accuracy en fonction du nombre d'époques

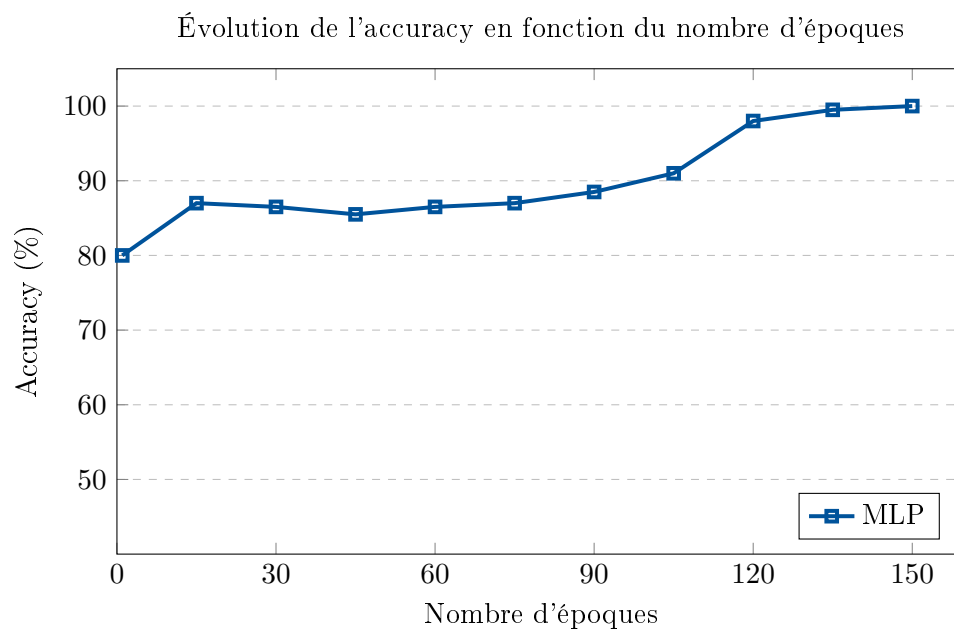


FIGURE 2 – Évolution de l'accuracy en fonction du nombre d'époques

Nous avons observé que le réseau atteint une performance optimale entre 105 et 120 époques.

5.2 Comparaison des différentes approches

Modèle	Accuracy
Frontière linéaire manuelle	$\approx 85\%$
Régression logistique (sklearn)	$\approx 87\%$
MLP codé manuellement (150 époques)	100%

TABLE 3 – Comparaison des performances des différentes approches

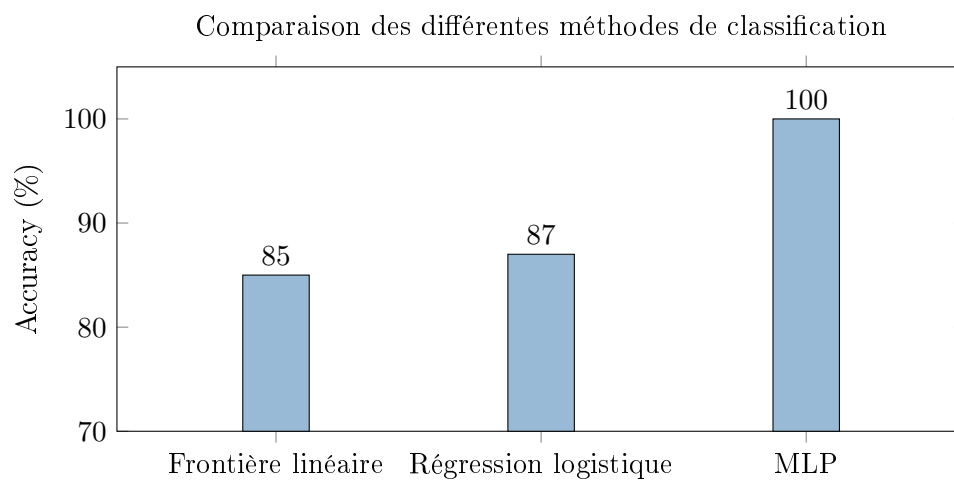


FIGURE 3 – Comparaison des performances des différentes approches

Cette comparaison montre clairement la supériorité du réseau de neurones pour ce problème de classification non linéaire. La capacité du MLP à modéliser des frontières de décision complexes lui permet d'atteindre une accuracy parfaite sur ce jeu de données.

6 Adaptation du modèle pour la classification du jeu MNIST

6.1 Adaptation de l'architecture

Le modèle initial était constitué d'une couche d'entrée avec 2 neurones, une couche cachée avec 10 neurones et une couche de sortie avec 2 neurones. Ce modèle était adapté à **Make_Moons**, qui présente deux classes séparées par une frontière simple. Cependant, pour **MNIST**, un jeu de données bien plus complexe, il a été nécessaire d'adapter l'architecture du réseau pour mieux gérer la classification de 10 classes différentes (les chiffres de 0 à 9).

Ainsi, la couche d'entrée a été modifiée pour accueillir 64 neurones, correspondant à une version réduite des images de 8x8 pixels extraites de MNIST (au lieu des 2 neurones initialement utilisés pour les données 2D de **Make_Moons**). La couche cachée a été étendue à 20 neurones pour mieux capturer les motifs non linéaires présents dans les données, et la couche de sortie a été ajustée pour inclure 10 neurones, correspondant aux 10 classes de chiffres du jeu **MNIST**.

6.2 Fonctions d'activation

Les mêmes fonctions d'activation ont été conservées par rapport au modèle initial : la couche cachée utilise la fonction d'activation **Sigmoïde**, et la couche de sortie utilise la fonction **softmax**.

6.3 Problèmes de performance et ajustements

Après avoir entraîné le modèle pendant 10 000 époques avec un taux d'apprentissage de 0.001, la précision obtenue était de seulement 0.006. Ce résultat révèle que, malgré l'architecture plus adaptée aux données **MNIST**, le modèle souffre d'un problème de **sous-apprentissage (underfitting)**. Plusieurs raisons expliquent cette faible performance :

- Le taux d'apprentissage choisi (0.001) pourrait être trop faible pour permettre une convergence efficace sur un jeu de données aussi complexe que **MNIST**, cette thèse est écartée car nous voyons bien que la loss diminue.
- Le nombre de neurones dans la couche cachée (20) pourrait être insuffisant pour capturer la complexité des images de chiffres manuscrits.
- Bien que 10 000 époques aient été utilisées, le modèle n'a peut-être pas eu suffisamment de capacité pour converger de manière efficace vers une solution optimale.

6.4 Comparaison avec le modèle sur **Make_Moons**

Pour **Make_Moons**, le modèle simple a obtenu 100% de précision, car le problème est bien plus simple. Les données et le modèle a pu facilement capturer les frontières de décision avec peu d'époques et une architecture légère. En revanche, **MNIST** est beaucoup plus complexe, nécessitant un modèle plus profond et plus de ressources pour capturer les variations subtiles des images des chiffres.

7 Pistes d'amélioration

Plusieurs pistes pourraient être explorées pour améliorer les performances de notre MLP :

Améliorations potentielles

- Augmenter la profondeur du réseau (ajouter plus de couches cachées)
- Augmenter le nombre de neurones dans les couches cachées
- Utiliser des fonctions d'activation plus performantes (ReLU, Leaky ReLU)
- Implémenter des techniques de régularisation (dropout, L2)
- Optimiser l'initialisation des poids (Xavier, He)
- Utiliser des algorithmes d'optimisation plus avancés (Adam, RMSprop)
- Appliquer des techniques de normalisation (Batch Normalization)
- Pour MNIST spécifiquement, utiliser une architecture convolutive (CNN)

8 Conclusion

Ce TP nous a permis de comprendre en profondeur les principes fondamentaux des réseaux de neurones artificiels. En implémentant un MLP à partir de zéro et en dérivant toutes les expressions mathématiques nécessaires pour l'algorithme de rétropropagation, nous avons acquis une compréhension approfondie du fonctionnement interne de ces modèles.

Les expérimentations ont confirmé la supériorité des réseaux de neurones pour les problèmes de classification non linéaires par rapport aux approches linéaires traditionnelles. Notre MLP a atteint 100% de précision sur le jeu de données `make_moons`, ce qui démontre son efficacité pour des problèmes de ce type. En revanche, lorsqu'il a été adapté pour le jeu de données MNIST, bien plus complexe, le modèle a montré des limites en raison de la simplicité de son architecture et de l'insuffisance de certains hyperparamètres. Cela a souligné l'importance de concevoir une architecture de réseau suffisamment robuste pour s'adapter à des jeux de données variés, et de bien ajuster les hyperparamètres (comme le taux d'apprentissage ou la taille de la couche cachée) afin d'optimiser les performances, comme l'illustre notre expérience avec MNIST.